

DeepGuard AI: Comprehensive Viva & Evaluation Guide

This document contains a comprehensive list of 50 potential questions that external examiners, technical interviewers, or team members might ask regarding the DeepGuard AI project. The questions are categorized into different aspects of the project, from high-level concepts to deep technical implementation details.

Part 1: Project Overview & Objectives (Questions 1-10)

1. What is the primary problem statement your project addresses? Answer: The primary problem is the rapid proliferation of highly realistic AI-generated images and deepfakes, which are increasingly being used for misinformation, fraud, and identity theft. While detection tools exist, almost all of them require users to upload their potentially sensitive images to remote servers for analysis. This creates a severe privacy vulnerability. Our project solves this by providing a comprehensive image intelligence platform that runs entirely client-side, within the user's web browser, ensuring zero data upload and complete privacy.

2. How does DeepGuard AI differ from existing solutions like Sensity AI or Microsoft Video Authenticator? Answer: The fundamental difference is the architecture. Existing solutions are server-centric or API-based; they process data in the cloud, which requires active internet connections, raises privacy concerns, and incurs ongoing server costs. DeepGuard AI is an "Edge AI" solution. We execute complex Vision Transformer models directly in the browser using WebAssembly and ONNX Runtime Web. This means no data ever leaves the user's device, ensuring absolute privacy, zero server costs for inference, and the ability to function offline once models are cached.

3. What are the five core tools provided by DeepGuard AI? Answer: The platform provides:

- 1. Deepfake Detection:** A binary classifier to detect facial manipulations.
- 2. AI Image Detection:** A 3-class classifier distinguishing between AI Art, Deepfakes, and Real photos.
- 3. Forensics Toolkit:** Browser-native tools for Error Level Analysis (ELA), Noise Analysis, RGB Histograms, EXIF extraction, and Integrity Scoring.
- 4. StegoCrypt:** A tool for hiding messages in images using LSB steganography combined with AES-256-GCM encryption.
- 5. Batch Scanning:** A utility to process multiple images simultaneously using the deepfake detection model.

4. Why did you choose to make this a web application rather than a native desktop or mobile app?

Answer: A web application provides universal accessibility without the friction of installation. Since our primary goal was to create a privacy-first tool, demonstrating that heavy AI inference and complex forensics can be done entirely within the browser's sandbox showcases the power of modern web technologies (WebAssembly, Web Crypto API, Canvas API). It ensures cross-platform compatibility (Windows, Mac, Linux, mobile) from a single codebase while still maintaining the security guarantees of client-side execution.

5. What is the significance of the "Zero-Upload Architecture"? Answer: It is the cornerstone of our security model. "Zero-upload" means that when a user analyzes an image, the image file is loaded only into the browser's local memory (RAM). The AI models are downloaded to the client and run locally. At no point is the image transmitted over HTTP/HTTPS to any external server. This guarantees that user data cannot be intercepted, logged, or misused by third parties.

6. Who is the target audience for DeepGuard AI? Answer: The target audience includes journalists verifying sources, security researchers analyzing manipulated media, legal professionals verifying evidence,

and general users concerned about the authenticity of images they encounter online or receive privately. The StegoCrypt feature also appeals to privacy advocates needing secure communication channels.

7. How do you handle the large size of AI models in a web application? Answer: This was a major technical challenge. We handle it through model quantization and caching. We use INT8 quantized versions of the Vision Transformer models, which reduces their size to around 50MB. When a user first uses a tool, the model is downloaded and permanently cached in the browser's IndexedDB. Subsequent visits load the model directly from this local cache in 1-3 seconds, completely bypassing the network.

8. What is the difference between the "Deepfake Detection" and "AI Image Detection" tools? Answer: The Deepfake Detection tool is a binary classifier specifically trained to identify facial manipulations (face swaps, reenactments) versus authentic human faces. The AI Image Detection tool is a broader, 3-class classifier designed to distinguish between entirely synthetic images (AI Art generated by Stable Diffusion/Midjourney), manipulated real images (Deepfakes), and entirely authentic photographs.

9. Can DeepGuard AI detect videos? Answer: Currently, the system is designed for static image analysis. Video detection is listed in our future scope. While the underlying models can process individual video frames, handling video decoding, frame extraction, and temporal analysis entirely client-side presents significant performance challenges that we plan to address in future iterations, potentially leveraging WebCodecs and WebGPU.

10. What role does Supabase play if the application is "zero-upload"? Answer: Supabase is strictly optional and used only for user authentication and managing scan history metadata. If a user logs in, the *results* of their scans (e.g., the verdict "Deepfake", the confidence score, the timestamp) are saved to the database to populate their dashboard. However, the image itself is *never* uploaded to Supabase. The core detection and forensic tools function perfectly without logging in.

Part 2: AI & Machine Learning Implementation (Questions 11-20)

11. What specific AI model architecture are you using for detection? Answer: We are using Vision Transformers (ViT). Specifically, the ViT-Base architecture with 16x16 patch embeddings. Unlike Convolutional Neural Networks (CNNs) which process images using local receptive fields, ViTs treat the image as a sequence of patches and use self-attention mechanisms to understand global relationships across the entire image. This makes them highly effective at detecting subtle inconsistencies introduced by AI generation or manipulation.

12. Which specific pre-trained models are deployed in the application? Answer:

1. For Deepfake Detection: `onnx-community/Deep-Fake-Detector-v2-Model-ONNX`
2. For AI Image Detection: `prithivMLmods/AI-vs-Deepfake-vs-Real-ONNX`

13. What is the accuracy of your Deepfake Detection model? Answer: The model achieves an overall accuracy of 92.12%. Breaking it down by class: it has a precision of 96.83% for detecting authentic images (Realism) and 87.40% precision for identifying manipulated images (Deepfakes). The F1-score is consistently high at ~91.7%.

14. Explain how ONNX Runtime Web facilitates client-side inference. Answer: ONNX (Open Neural Network Exchange) is an open standard format for ML models. ONNX Runtime Web is a JavaScript library that allows these models to be executed in the browser. It works by compiling the execution engine into WebAssembly (WASM). When we pass an image tensor to the runtime, it executes the mathematical operations of the neural network using the CPU via WASM, heavily optimized with SIMD (Single Instruction, Multiple Data) instructions, entirely within the client's environment.

15. Why did you choose Transformers.js (v3) over other libraries like TensorFlow.js? Answer: Hugging Face's `@huggingface/transformers` (v3) provides a significantly more streamlined developer experience for transformer architectures compared to TensorFlow.js. It handles the complex pipeline seamlessly: downloading the model weights from the Hugging Face Hub, managing the IndexedDB caching, running the specific image preprocessors (resizing, normalization) required by the model, and executing the inference via its bundled ONNX Runtime Web. Version 3 specifically added support for ONNX IR v10, which was necessary for our newer ViT models.

16. What preprocessing steps are applied to an image before inference? Answer: The image is first decoded and resized to the specific input dimensions expected by the ViT model, which is 224x224 pixels. It is then converted into a tensor and normalized. The normalization involves scaling the pixel values (typically 0-255) and applying specific mean and standard deviation values derived from the dataset the model was originally trained on (usually ImageNet stats).

17. How does the 3-class AI Image Detection model arrive at its verdict? Answer: The ViT model outputs raw logits for the three classes (Artificial, Deepfake, Real). We apply a softmax function to these logits to convert them into probability percentages that sum to 100%. The class with the highest probability is selected as the verdict. The UI then displays these probabilities as the "Confidence Breakdown" bars.

18. What happens if the browser doesn't have enough memory to load the model? Answer: This is a known limitation of client-side AI. The ViT models require approximately 200MB of RAM during inference. If a device is severely resource-constrained (e.g., an older low-end mobile phone), the browser tab might crash or throw an Out Of Memory error. We mitigate this by using INT8 quantized models to keep the footprint as small as possible.

19. How did you resolve the ONNX IR version mismatch issue during development? Answer: We initially used `@xenova/transformers` (v2), which bundled an older version of `onnxruntime-web` supporting up to ONNX IR version 8. The AI Image Detection model we selected was exported using IR version 10, causing a load failure. We resolved this by migrating to the official successor package, `@huggingface/transformers` (v3), which includes an updated ONNX runtime capable of parsing the newer model format.

20. Does the model learn or update itself based on user uploads? Answer: No. The models deployed are static, pre-trained ONNX models. Because the application operates entirely client-side with zero data upload, we have no mechanism to collect user images for retraining. While this prevents the model from learning continuously from user data, it is a necessary trade-off to guarantee absolute privacy. Implementing federated learning is a potential future scope.

Part 3: Image Forensics & Analysis (Questions 21-30)

21. What is Error Level Analysis (ELA) and how is it implemented? Answer: ELA identifies image manipulation by detecting differences in JPEG compression levels. When an image is modified and saved, the edited region often has a different compression artifact signature than the original area. We implement this using the HTML5 Canvas API: we render the original image, export it as a JPEG at 95% quality, load that recompressed image back into a new canvas, and calculate the absolute pixel-by-pixel difference between the original and recompressed versions. We amplify this difference (e.g., by a factor of 20) to make it visually obvious; manipulated areas appear as bright spots against a darker, uniform background.

22. How does the Noise Residual Analysis work? Answer: Every camera sensor or generation process leaves a specific noise pattern. Splicing images together disrupts this uniformity. We isolate the noise by applying a 5x5 box blur to the image and subtracting the blurred result from the original image. The residual

difference represents the high-frequency noise. We then amplify this noise and map it to an "inferno" colormap to visually highlight inconsistencies across different regions of the image.

23. What can an RGB Histogram tell us about an image's authenticity? Answer: An RGB Histogram plots the frequency of pixel intensities (0-255) for the Red, Green, and Blue channels. Authentic photographs usually have complex, jagged histograms due to natural lighting and sensor noise. AI-generated images, especially those from older GANs or heavily processed outputs, sometimes exhibit unnaturally smooth distributions or unnatural spikes (clipping) because they are mathematically generated rather than optically captured.

24. Which library is used for EXIF extraction, and what data is retrieved? Answer: We use the `exifr` library because it operates entirely client-side and supports parsing TIFF, EXIF, GPS, ICC, IPTC, and XMP data blocks. We extract metadata such as camera make/model, original date/time, ISO speed, exposure time, focal length, color space, and GPS coordinates if available.

25. How do you detect AI generation software using EXIF data? Answer: When AI tools like Midjourney or Stable Diffusion generate an image, they often write their name into the EXIF `Software` or `Creator` tags. Our EXIF extraction module checks these tags against a predefined list of known AI indicators (e.g., "midjourney", "stable diffusion", "comfyui", "automatic1111"). If a match is found, the image is immediately flagged.

26. What happens if an image has had its EXIF data stripped? Answer: This is very common, especially on social media platforms which strip metadata for privacy reasons. Our Integrity Scoring system accounts for this. An image with little or no EXIF data is penalized in the score because a lack of provenance information increases risk. However, the visual forensics (ELA, Noise, AI models) do not rely on EXIF data and will still function normally on stripped images.

27. Explain the algorithm behind the "Integrity Score". Answer: The Integrity Score is a weighted heuristic starting at 100. Points are deducted based on various risk factors:

- Lack of EXIF data (-25 points for <3 fields, -10 for 3-9 fields).
- Missing camera information (-10 points).
- Detection of AI software tags (-40 points).
- Missing GPS or Date/Time data (-5 points each).
- High variance in noise patterns across image quadrants (up to -15 points). The final score is clamped between 0 and 100 and mapped to Risk Levels: Low (≥ 70), Medium (40-69), High (<40).

28. How is the noise variance calculated for the Integrity Score? Answer: The image is divided into four quadrants. We calculate the standard deviation of the pixel intensities (the noise residual) within each quadrant. We then find the difference between the maximum and minimum standard deviations across the four quadrants. A high variance indicates that some parts of the image have very different noise characteristics than others, strongly suggesting splicing or manipulation.

29. Why use the Canvas API instead of an external image processing library? Answer: Using the native HTML5 Canvas API aligns perfectly with our zero-dependency, client-side philosophy. It allows us to perform pixel-level manipulations (like ELA and blurring) directly in the browser's rendering engine without needing to compile complex C++ libraries (like OpenCV) to WebAssembly, keeping the application lightweight and fast.

30. What is the purpose of computing a SHA-256 hash in the forensics toolkit? Answer: The SHA-256 hash acts as a cryptographic fingerprint for the exact file uploaded. It provides a mathematical guarantee of the file's state at the time of analysis. If a user needs to prove that a specific analysis report corresponds to a specific image file, the hash serves as that immutable link.

Part 4: Cryptography & Steganography (Questions 31-40)

31. What is Steganography, and how is it used in StegoCrypt? Answer: Steganography is the practice of concealing a file, message, image, or video within another file, message, image, or video. In StegoCrypt, we use it to hide secret text messages inside standard image files. While encryption protects the *contents* of a message, steganography hides the *existence* of the message entirely.

32. Explain the LSB (Least Significant Bit) encoding technique. Answer: Digital images are composed of pixels, typically represented by Red, Green, and Blue (RGB) values ranging from 0 to 255 (8 bits per channel). LSB steganography involves replacing the least significant bit (the 1st bit, which has the smallest impact on the numerical value) of these color channels with the bits of our secret message. Because changing the LSB only alters the color value by a maximum of 1 unit, the visual change is imperceptible to the human eye.

33. How does the system know where the hidden message ends during decryption? Answer: Before encoding the message into binary, we append a specific delimiter string, specifically `<<END>>`. During the extraction process, the system continuously converts the extracted bits back into text characters and checks the end of the string for this delimiter. Once `<<END>>` is detected, it stops processing and returns the message, discarding the rest of the image data.

34. Why do you use the Web Crypto API instead of a JavaScript cryptography library? Answer: The Web Crypto API (`window.crypto.subtle`) is built natively into modern browsers. It is faster, more secure, and often backed by the operating system's hardware cryptographic modules. Using external JavaScript libraries (like CryptoJS) introduces performance overhead and potential security vulnerabilities, whereas the native API provides FIPS-compliant implementations of standard algorithms.

35. Detail the encryption process used in StegoCrypt. Answer:

- 1. Key Derivation:** The user's password is run through the PBKDF2 algorithm along with a salt ("stego-salt-v1"), using SHA-256 and 100,000 iterations to derive a secure 256-bit key.
- 2. Encryption:** The text message is encrypted using the AES-GCM (Advanced Encryption Standard in Galois/Counter Mode) algorithm with the derived key and a randomly generated 12-byte Initialization Vector (IV).
- 3. Packaging:** The 12-byte IV is prepended to the resulting ciphertext.
- 4. Encoding:** This combined byte array is then base64 encoded and passed to the LSB steganography module for embedding.

36. Why is PBKDF2 used with 100,000 iterations? Answer: PBKDF2 (Password-Based Key Derivation Function 2) is used to stretch a user-provided password into a cryptographically strong key. Running it through 100,000 iterations makes the process deliberately slow and computationally expensive. This is a crucial defense against brute-force or dictionary attacks, making it practically infeasible for an attacker to guess the password by rapidly trying millions of combinations.

37. What is the significance of the Initialization Vector (IV) in AES-GCM? Answer: The IV ensures that even if the exact same message is encrypted twice with the exact same password, the resulting ciphertext will be completely different. This prevents attackers from identifying patterns. AES-GCM requires the IV to be unique for every encryption operation under the same key. We generate a random 12-byte IV for every encryption using `crypto.getRandomValues`.

38. Can the hidden message survive if the image is uploaded to WhatsApp or Instagram? Answer: No. LSB steganography relies on exact, lossless pixel values. Social media platforms and messaging apps heavily compress images (usually converting them to lossy JPEG format) to save bandwidth. This

compression permanently alters the pixel values, destroying the fragile LSB data. StegoCrypt images must be transmitted via lossless channels (e.g., email attachments, cloud storage links, or ZIP files).

39. What is the maximum message capacity of the StegoCrypt tool? Answer: Since we use the RGB channels (skipping the Alpha channel), we have 3 bits available per pixel. Each character requires 8 bits. Therefore, the capacity in characters is roughly $(\text{Width} \times \text{Height} \times 3) / 8$. For a standard 1080p image (1920x1080), this allows hiding approximately 777,600 characters, which is more than enough for extensive text documents.

40. If I don't enter a password, is the message still secure? Answer: No. If no password is provided, the system only uses LSB steganography without AES encryption. The message is *hidden*, but anyone who knows it is a steganographic image and uses a standard LSB extraction tool can read the plaintext message. The AES encryption is what provides actual cryptographic security.

Part 5: Web Engineering & Architecture (Questions 41-50)

41. Describe the overall technology stack of the application. Answer: The frontend is built using React 18 and TypeScript, bundled with Vite. Styling is handled by Tailwind CSS and the shadcn/ui component library. Animations use Framer Motion and CSS keyframes. State management and routing are handled by React Query and React Router DOM. The core AI logic relies on `@huggingface/transformers` and ONNX Runtime Web. Authentication and database services (optional) are provided by Supabase. The application is hosted statically on Netlify.

42. What is the "Aurora Theme" and how is it implemented? Answer: The Aurora Theme is our custom design system. It features a dark slate background (`#1e293b`) with vibrant gradient accents of emerald (`#10b981`), teal (`#14b8a6`), and sky blue (`#0ea5e9`). We implemented it primarily using CSS variables defined in `index.css`, which are then utilized by Tailwind utility classes. It heavily features glassmorphism (translucent backgrounds with `backdrop-blur`), soft glowing shadows, and gradient text.

43. How do you handle complex animations without impacting performance? Answer: We separate concerns. Complex layout animations, orchestrations, and scroll-triggered reveals are handled by **Framer Motion**, which uses `requestAnimationFrame` and optimized DOM updates. Continuous, ambient background animations (like the drifting Aurora blobs, the flowing gradient bar, or the floating particles) are implemented using **pure CSS keyframes**. CSS animations run on the browser's compositor thread, often leveraging GPU acceleration, which ensures they remain smooth without blocking the main JavaScript thread where our AI inference runs.

44. What was the purpose of creating the `PageTransition` component? Answer: The `PageTransition` component wraps our React Router routes using Framer Motion's `AnimatePresence`. It ensures that when a user navigates between tools, the current page fades out smoothly before the new page fades in. This prevents jarring layout shifts and contributes significantly to the premium, professional feel of the application.

45. Explain the purpose of `vite.config.ts` in your project. Answer: Vite is our build tool. The configuration file defines how the application is compiled. Key configurations include setting up path aliases (e.g., mapping `@/` to the `src/` directory for cleaner imports), configuring the React SWC plugin for faster compilation, and specifying build targets. It also handles dependency pre-bundling, which was crucial for resolving cache issues we encountered with the ONNX runtime dependencies.

46. How did you resolve the React Router "Rendered more hooks than during the previous render" error? Answer: This is a classic React error that occurs when hooks (like `useState` or `useEffect`) are called conditionally or after an early return statement. React relies on the exact order of hook calls to

maintain state between renders. We fixed this by reviewing the components (specifically the Protected Routes and Dashboard areas) and ensuring that all hooks were declared at the very top level of the functional component, before any conditional logic or `return` statements based on authentication state.

47. Describe the structure and utility of the `shadcn/ui` components. Answer: `shadcn/ui` is not a traditional component library that you install via npm. Instead, it provides beautifully designed, accessible, and customizable components (built on top of Radix UI primitives) that you copy and paste into your project's source code (usually under `src/components/ui/`). This gives us complete control over the styling and behavior of elements like Buttons, Cards, Dialogs, and Progress bars, allowing us to seamlessly integrate them into our custom Aurora theme without fighting overriding CSS specificity.

48. Why do you use both `Toaster` from `shadcn` and `Sonner` ? Answer: They serve slightly different notification purposes. We primarily transitioned to using `Sonner` because it provides a more modern, unobtrusive toast notification system with better support for promises, complex layouts, and swipe-to-dismiss functionality, which improves the UX during long-running tasks like model downloading or complex forensic processing.

49. How is the Batch Scanning feature architected to prevent freezing the UI? Answer: Analyzing multiple images consecutively is computationally heavy. If run synchronously, it would lock the main thread, freezing the browser UI. To prevent this, the BatchScan component manages an asynchronous queue. It processes images iteratively using `for...of` loops with `await`. While the underlying WASM execution is synchronous within its thread, yielding execution back to the React event loop between images allows the UI to update progress bars and render partial results without freezing.

50. What is the deployment strategy for DeepGuard AI? Answer: DeepGuard AI is a purely static Single Page Application (SPA). Running `npm run build` generates a `dist` folder containing optimized HTML, CSS, and JavaScript. This folder is deployed to Netlify, a global Content Delivery Network (CDN). Because there is no backend server required for the core application (Supabase is accessed directly via client-side API calls, and Hugging Face provides the model CDN), the application is highly scalable, highly available, and extremely fast to load globally.